

GUÍA DE ESTUDIO: COLECCIONES EN PYTHON (UT 3.1)

Autor: JAPR

Fecha: 18/02/26

Objetivo: Examen Python

1. CUADRO COMPARATIVO (Resumen Rápido)

Esta tabla resume las características principales para elegir la colección adecuada.

Colección	Símbolo	Modelo Mental	¿Ordenado?	¿Mutable? (Cambiar)	¿Permite Duplicados?
LISTA	[]	Mochila (Cabe todo)	SÍ	SÍ	SÍ
TUPLA	()	Caja Fuerte (Blindada)	SÍ	NO	SÍ
SET	{ }	Filtro (Borra repetidos)	NO	SÍ	NO
DICCIONARIO	{k:v}	Agenda (Clave/Valor)	SÍ (Python 3.7+)	SÍ	Claves NO / Valores SÍ

2. LISTAS (list)

Uso: La herramienta principal. Úsala cuando necesites guardar datos en orden y vayas a añadir, borrar o modificar elementos posteriormente.

- **Crear:** `numeros = [1, 2, 3, 4]`
- **Acceder:**
 - `numeros[0]` (El primer elemento)
 - `numeros[-1]` (El último elemento)
 - `numeros[1:3]` (Slicing: recorta un subconjunto)

Métodos Clave

- `.append(50)`: Añade el elemento al final de la lista.
- `.pop()`: Elimina y devuelve el último elemento (o el índice especificado).

- `.remove(valor)`: Busca el valor y borra la primera coincidencia.
- `.sort()`: Ordena la lista modificando la original.
 - *Nota*: Si quieres ordenar sin modificar la original, usa `sorted(lista)`.

Ejemplo: Eliminar el elemento central

Python

```
indice_medio = (len(lista) - 1) // 2
lista.pop(indice_medio)
```

3. TUPLAS (tuple)

Uso: Seguridad e integridad de datos. Úsala para datos fijos (coordenadas, días de la semana, configuraciones) que no deben ser modificados por error.

- **Crear:** `punto = (10, 20)`
- **Acceder:** Igual que las listas (`punto[0]`).
- **Inmutabilidad:** Si intentas `punto[0] = 50`, Python lanzará un error.

Desempaquetado (Unpacking)

Permite asignar múltiples variables en una sola línea:

Python

```
datos = ("Ana", 25)
nombre, edad = datos
# Resultado: nombre="Ana", edad=25
```

4. SETS (Conjuntos)

Uso: Limpieza de datos y operaciones de conjuntos. Úsalo para eliminar duplicados instantáneamente o comparar grupos de datos.

- **Crear:** `colores = {"rojo", "azul"}` o `set(lista_sucia)`
- **Importante:** No tienen índice. No se puede acceder mediante `colores[0]`.

Operaciones Matemáticas

- **Unión (|):** Combina todos los elementos de ambos conjuntos.
- **Intersección (&):** Muestra solo los elementos comunes.
- **Diferencia (-):** Muestra los elementos del primer conjunto que no están en el

segundo.

Ejemplo: Limpiar duplicados de una lista

Python

```
lista_limpia = list(set(lista_con_duplicados))
```

5. DICCIONARIOS (dict)

Uso: Estructura de datos clave-valor. Úsalo para representar objetos complejos o realizar conteos de frecuencias.

- **Crear:** `alumno = {"nombre": "Luis", "nota": 8}`
- **Acceder:** `alumno["nombre"]`
- **Añadir/Modificar:** `alumno["nota"] = 9` (Si la clave existe, la actualiza; si no, la crea).

Recorrer un diccionario

Para el examen, se recomienda usar `.items()` para obtener clave y valor simultáneamente:

Python

```
factura = {"Monitor": 200, "Ratón": 20}
```

```
for producto, precio in factura.items():  
    print(f"El {producto} cuesta {precio} euros")
```

Lógica del Contador (Algoritmo Clave)

Patrón estándar para contar frecuencias (votos, palabras, etc.):

Python

```
if palabra in diccionario:  
    diccionario[palabra] += 1 # La clave existe, incrementamos el valor  
else:  
    diccionario[palabra] = 1 # Es nueva, inicializamos a 1
```

6. ERRORES COMUNES Y CONSEJOS

1. **Entrada de datos:** La función `input()` siempre devuelve texto (string).

- Si necesitas números: usa `int(input())` o `float(input())`.
- Si vas a comparar texto: normaliza con `.lower()` y limpia espacios con

.strip()).

2. **Copias de listas:** lista2 = lista1 no crea una copia, sino una referencia. Si modificas lista2, cambiará lista1. Para duplicar, usa lista1.copy().
3. **Búsqueda en diccionarios:** Acceder a una clave inexistente provoca un error. Usa el método .get("clave", "Valor por defecto") para evitar fallos.

4. Atajos VS Code:

- Formatear código: Shift + Alt + F
- Limpiar consola: Ctrl + L

7. EJEMPLOS PRÁCTICOS RESUELTOS

A. Procesar Lista (Añadir, Ordenar, Eliminar centro)

Python

```
def procesar_lista(lista, nuevo_num):
    # Verificación de longitud
    if len(lista) != 5:
        return "Error: Longitud incorrecta"

    lista.append(nuevo_num)    # 1. Añadir
    lista.sort()              # 2. Ordenar

    # Cálculo del índice central
    medio = (len(lista) - 1) // 2
    lista.pop(medio)          # 3. Eliminar centro

    return lista
```

B. Análisis de Factura (Diccionarios)

Python

```
factura = {"Monitor": 299.50, "Ratón": 29.99}

# Sumar total (usando .values())
total = sum(factura.values())

# Búsqueda segura
precio_webcam = factura.get("Webcam", "No disponible")
```